LEGAL INTELLIGENCE PLATFORM

TECHNICAL SPECIFICATION DOCUMENT | TSD v1.1

OTIC GROUP • OCTOBER 2026

Enterprise architecture • Data & integration contracts • Agentic AI orchestration • Security • Deployment • Engineering acceptance

Status: Proposed technical baseline for engineering and institutional validation. Vendor choices, quantitative service targets, hosting region and external source licences require approval.

Document control

Item	Value
Document	Legal Intelligence Platform Technical Specification Document
Version	1.0
Source baseline	Master BRD v2.1 and FSD v1.1
Scope	Law firms and corporate legal departments
Owner	Otic Group — Product and Engineering
Status	Design proposal; not an approved production architecture

1. Executive architecture decision

LIP is a multi-tenant enterprise legal intelligence system with four distinct application environments: customer-facing interaction; internal institutional intelligence; AI Legal Command Centre; and Otic platform administration. These environments share policy-governed services, workflow orchestration and auditable data infrastructure. Tenant legal content remains isolated from Otic commercial administration.

Plane	Core services	Trust boundary
Experience	WhatsApp gateway, client portal, institutional web app, Command Centre UI, Otic admin UI	Separate authentication, sessions and presentation permissions
Application	Intake, CRM, conflicts, matters, documents, research, contracts, litigation, compliance, billing, analytics	Tenant-scoped APIs and domain authorisation
Intelligence	Command parser, policy engine, tool registry, agent orchestration, retrieval, evaluation	Permission-filtered context; approval gates
Data	Transactional store, object storage,	Tenant-scoped encryption,

	search/vector index, event stream, analytical projections	isolation, audit
Platform	Identity, observability, metering, billing, CI/CD, backups, operations	Administrative plane cannot browse tenant content

2. Logical component and request flows

Client channel → Channel adapter → Inbound signature/consent validation → Identity resolver → Intake workflow → Event bus → Unified inbox / matter services → Dashboard projections. Internal user → Identity + policy decision → Command Centre → intent/tool plan → scoped retrieval or action preview → human approval where required → tool execution → audit receipt.

Architectural recommendation: modular service boundaries with an event-driven integration layer. A modular monolith may be used for the initial MVP if strict tenant and domain boundaries are enforced; extraction to services should be driven by measured scale and reliability needs.

3. Environment and deployment topology

Environment	Isolation and controls	Release policy
Development	Synthetic legal data; sandbox integrations; non-production keys	Automated build and tests
Staging / UAT	Production-like configuration, test tenant separation, masked data only if approved	Business, security and legal acceptance
Production	Private network tiers, managed secrets, WAF, restricted administrative access	Approved change window, rollback, monitored deployment
Disaster recovery	Encrypted backups, restoration runbooks, regional design subject to residency approval	Scheduled restore exercises and sign-off

Cloud provider, region, managed services, private connectivity, sovereignty requirements and production topology are decisions to be documented by the institution and Otic engineering. No specific cloud is mandated by this TSD.

4. Identity, tenancy and authorisation

Control	Implementation contract
Tenant resolution	Resolve tenant from verified hostname/identity context; never trust a user-supplied tenant_id alone
Authentication	OIDC/OAuth2 for institutional web access; MFA for privileged users; verified OTP for permitted client actions
Authorisation	Central policy decision point with RBAC plus ABAC (tenant, matter, role, ethical wall, document classification, action)

Data isolation	Every query, object key, index and analytics projection includes enforced tenant scope; automated cross-tenant tests
Privileged access	Just-in-time administrative elevation; segregation of duties; all access logged
Customer channels	Bind WhatsApp/web identities only after approved verification; store consent and verification evidence

5. Domain data model

Entity	Primary identifiers and relationships	Key integrity rule
Tenant	tenant_id; institution_mode; plan_id	Root of all institution-owned records
User / membership	user_id; tenant_id; role_id; policy_attributes	Unique membership and active status
Client / contact	client_id; tenant_id; verified channel identities	Merge reviewed and audit-preserved
Enquiry / conversation	enquiry_id; conversation_id; channel_event_id; client_id	Origin channel immutable; event ID unique per provider
Matter / party / conflict	matter_id; client_id; party_id; conflict_check_id	Active matter requires clearance/engagement state
Document / version	document_id; version_id; object_hash; matter_id	Immutable version lineage; malware scan before availability
Authority / citation	source_id; jurisdiction; effective_date; licence_ref	Source currency and citation verification
AI run / tool call	run_id; context_id; tool_call_id; model_version	Reproducible metadata; bounded retention
Approval / audit	approval_id; actor_id; decision; timestamp; policy_version	Append-only event history
Usage / invoice	usage_id; tenant_id; units; price_version; invoice_id	Idempotent consumption and financial reconciliation

Implementation recommendation: relational transactional database for authoritative business records, object storage for documents, search/vector indexes for permitted retrieval, and event-driven read models for dashboards. Index entries must carry tenant, classification and access-control metadata.

6. API conventions and contracts

Area	Normative technical convention
Transport	HTTPS; versioned JSON REST APIs for transactional operations; signed webhooks for inbound events
Identity	Short-lived bearer tokens with audience, issuer and tenant membership validation

Idempotency	Idempotency-Key on state-changing external requests; provider_event_id uniqueness for webhook processing
Concurrency	Optimistic version/ETag checks for matter, document and approval updates
Pagination	Cursor-based pagination; stable ordering; maximum page sizes configured
Errors	Stable error codes, correlation_id, safe user message, retryable flag; no privileged details in errors
Observability	Trace/correlation IDs across API, workflow, AI run, events and outbound channel messages
Compatibility	Explicit API version and deprecation policy; schema validation and contract tests

7. Illustrative endpoint inventory

Method / path	Purpose	Access rule
POST /v1/enquiries	Create validated client enquiry	Channel service; verified consent
GET /v1/inbox	Institutional conversation queue	Tenant and queue membership
POST /v1/clients/{id}/verify	Bind verified client channel identity	Identity service and approved workflow
POST /v1/matters	Open matter after clearance gates	Matter creation permission
POST /v1/conflicts/checks	Screen related parties and matters	Conflict-screening permission
POST /v1/documents	Create document/upload session	Matter access; scan and classification
POST /v1/commands/runs	Submit scoped natural-language command	Command permission and allowed tools
POST /v1/approvals/{id}/decisions	Approve or reject proposed action	Named approver; segregation of duties
GET /v1/analytics/metrics	Return governed metric results	Field and cohort-level analytics permissions
GET /v1/admin/tenants	Manage tenant commercial configuration	Otic operations; no legal payloads

These are reference endpoint shapes, not a frozen OpenAPI contract. The implementation team must publish OpenAPI schemas, validation examples, response codes, pagination rules and generated contract tests before development sign-off.

8. Messaging and omnichannel integration

Step	Technical behaviour
Receive	Verify provider signature and timestamp; reject replay; persist raw event metadata securely
Normalise	Map provider payload to canonical message envelope

	with tenant, channel, event and conversation IDs
Resolve	Apply verified identity match; uncertain matches remain provisional and restricted
Route	Publish event to intake workflow; create inbox projection; assign queue/owner by configured policy
Reply	Validate consent/template/channel rules; queue outbound message; record provider delivery status
Recover	Retry transient failures with exponential backoff; dead-letter poison events; reconcile delivery state

9. Agentic AI Command Centre — execution architecture

The Command Centre is a policy-governed orchestration service rather than an unrestricted chatbot. Every run must be bound to the authenticated actor, tenant, explicit matter context where applicable, and allowed tools.

Stage	Required behaviour	Audit output
1. Parse	Classify retrieval, analysis, generation or execution; clarify ambiguous context	Intent and scope
2. Authorise	Check tenant, role, matter, ethical wall, classification and allowed tools	Policy decision
3. Plan	Select specialised agent(s), tool schemas, budgets and stop conditions	Versioned plan
4. Retrieve	Search only approved and permission-filtered legal/institutional sources	Source IDs and freshness
5. Execute	Call validated tools; enforce timeouts, budgets and retry ceilings	Tool inputs/outputs metadata
6. Verify	Check structured schema, calculations, citations, jurisdiction and unsupported assertions	Validation flags
7. Approve	Present proposed consequential action and obtain authorised human approval	Approval decision
8. Commit	Execute idempotently; return result, receipt and state changes	Execution receipt
9. Evaluate	Capture user feedback, error class, quality and cost metrics	Evaluation record

10. Specialised agents and tool boundaries

Agent	Permitted tools	Mandatory checks

Legal research	Approved authority search, citation resolver, jurisdiction filters	Court/source authority, effective date, source licence, no fabricated citation
Document drafting	Approved templates, matter files, clause library, versioning	Draft watermark/status, review before release
Contract intelligence	Clause extraction, comparisons, obligation tracker	Version traceability and human review
Litigation	Matter timeline, evidence index, deadline tools	Chain of custody, court rules, human validation
Compliance	Obligation register, regulatory source monitor	Effective date and responsible reviewer
Operations and analytics	Read-only governed metrics, permitted work queues	Deterministic calculations, cohort definition
Workflow agent	Assignment, task creation, approved notification actions	Preview, role check, approval and idempotent execution

11. Retrieval-augmented generation and legal source governance

- Ingestion pipeline: source licence check → malware/file validation → extraction → metadata enrichment → jurisdiction/effective-date tagging → chunking → indexing → quality review.
- Retrieval: tenant and matter permission filters are applied before context reaches the model; avoid post-hoc redaction as the sole control.
- Citation: link each substantive legal claim to identifiable authorities and passage references where available; label unverified or secondary sources.
- Freshness: record indexed_at, source_updated_at, effective_from/effective_to, verification status and refresh owner.
- Prompt injection: treat documents, websites, messages and tool outputs as untrusted; tools execute only through explicit policy checks.
- Knowledge separation: institution-private precedents and opinions must never leak into another tenant or public retrieval corpus.

12. Approval and transaction model

Proposed actions use a two-phase pattern: prepare/preview and approve/commit. Approval records bind the actor, action type, target resource, proposed payload hash, policy version, expiry and approver. A modified action invalidates prior approval. Executed actions return an immutable receipt and correlated audit events.

13. Dashboards, analytics and data quality

Technical concern	Design requirement
Data lineage	Each KPI maps to authoritative source tables/events, transformation version and documented formula
Eventual consistency	Expose last_refreshed_at and lag; dashboard reconciliation jobs compare projections with source-of-truth
Attribution	Immutable first-touch channel plus separately logged

	corrected attribution and cohort rules
Metrics	Qualified rate, lead-to-matter conversion, SLA, matter cycle time, collections and AI unit costs calculated deterministically
Drilldowns	Same permission filters on aggregates and underlying records; protect against small-group inference where necessary
Exports	Audited, time-limited, permission-checked downloads; watermark where institution requires

14. Document and evidence storage

- Store objects under tenant- and matter-scoped namespaces with encrypted storage and controlled signed access URLs.
- Require file type validation, malware scanning, checksum, version identifier and uploader provenance.
- Support retention schedules, legal holds and defensible deletion approvals without breaking immutable audit history.
- Record chain-of-custody events for evidentiary material; distinguish original, derived, annotated and AI-generated artifacts.

15. Security and privacy engineering

Layer	Baseline control
Network	TLS, WAF, private service endpoints where available, least-privilege ingress/egress
Secrets	Managed vault; rotation; no credentials in code, logs or prompts
Encryption	At rest and in transit; tenant key strategy and key custody subject to risk review
Privacy	Data minimisation, lawful basis/consent, retention, export/deletion request handling
AI security	Prompt injection controls, tool allowlists, output validation, sensitive-data minimisation
Audit	Tamper-evident logs, clock synchronisation, actor/action/resource correlation
Software supply chain	Dependency scans, SBOM, signed artifacts where feasible, vulnerability triage
Assurance	Threat modelling, penetration testing, incident response drills before production

16. Non-functional service level framework

Dimension	Proposed verification approach	Decision status
Availability	Define service tier, exclusions,	Target to approve

	monitoring windows and monthly calculation	
Response latency	Specify p95/p99 for portal, search, analytics and AI runs separately	Target to approve
Capacity	Load test concurrent users, messages/minute, matters, document sizes and AI runs	Sizing to approve
Resilience	Define RTO/RPO, backup intervals, recovery drills and dependency degradation	Targets to approve
Data retention	Specify per data class, jurisdiction, legal hold and deletion policy	Policy to approve
AI cost	Per-tenant budgets, quotas, model routing, token metering and alerts	Commercial policy to approve

17. Observability, incident and cost operations

Instrument service metrics, distributed traces, structured logs, queue depth, provider delivery rates, model latency, retrieval hit/verification rates, approval delays, error budgets, tenant consumption and margin. Dashboards must avoid storing confidential legal prompts in general operational logs.

Incident severity, paging routes, security notification, customer communication, rollback, root-cause analysis and post-incident action ownership require formal runbooks.

18. Commercial administration and metering

Otic administration is a separate trust domain. Usage events record tenant, capability, billable unit, price version, timestamp and deduplication key. Invoice calculation must reconcile metering events and approved pricing. The administration console can inspect aggregated service and commercial metadata, not confidential client/matter content.

19. CI/CD and engineering standards

- Infrastructure as code with environment-specific configuration and policy checks.
- Pull request review, linting, unit tests, API schema tests, SAST/dependency scanning and secret detection.
- Migration discipline: backward-compatible schema changes, verified backups and rollback/roll-forward plans.
- Feature flags and tenant-level entitlements for staged rollout; avoid untested production toggles.
- Separate non-production test fixtures from live client data; prohibit live privileged content in demos.

20. Test strategy and acceptance gates

Test layer	Required evidence
Unit / component	Domain rules, validators, policy decisions, KPI calculations, state transitions
Contract / integration	OpenAPI schemas, webhook signature/replay,

	provider errors, retries, idempotency
End-to-end	WhatsApp → intake → inbox → conflict clearance → matter → dashboard
AI evaluation	Citation correctness, refusal/uncertainty, jurisdiction accuracy, prompt-injection and tool abuse
Security	Cross-tenant leakage, ethical wall, role escalation, export controls, audit integrity
Resilience	AI outage, messaging outage, restore test, queue replay and duplicate events
UAT	FSD v1.1 UAT-21 to UAT-32 plus full BRD-to-test traceability

21. Delivery work packages

Package	Core outputs	Dependency
WP1 Platform foundation	Tenancy, identity, policy engine, audit, environments	Security and hosting decisions
WP2 Customer intake	Channel adapter, identity, consent, conversation lifecycle	Messaging provider configuration
WP3 Legal operations	Clients, conflicts, matters, documents, tasks	Institutional workflow sign-off
WP4 Intelligence and Command Centre	Agents, RAG, tools, approvals, evaluation	Licensed sources and policy engine
WP5 Dashboards and commercials	KPI models, projections, metering, billing	Metric definitions and pricing approval
WP6 Assurance and rollout	UAT, performance, security, DR, operational handover	Approved acceptance targets

22. Architecture decision register

ADR	Decision needed	Recommended default / trade-off
ADR-01	Hosting provider and data region	Select on residency, contractual, resilience and operating cost requirements
ADR-02	Service decomposition	Start modular, isolate bounded contexts; extract services when justified
ADR-03	Tenant isolation level	Enforced tenant scoping plus stronger physical separation for high-risk customers if required
ADR-04	LLM provider and fallback	Pluggable model gateway; approved provider list and no unapproved training use
ADR-05	Legal source licensing	Official/authorised sources and

		documented commercial reuse rights
ADR-06	Search and vector technology	Permission-aware indexes; test recall, isolation and operating cost
ADR-07	RTO/RPO, SLO and retention	Agree numeric thresholds and fund matching architecture
ADR-08	Billing and price book	Versioned units and reconciliation controls

23. BRD / FSD technical traceability

Business capability	FSD baseline	TSD implementation sections
Customer-facing platform	Omnichannel lifecycle and screen inventory	2, 4, 6–8
Internal intelligence	Inbox, matter, analytics, permissions	4–6, 13–14
AI Command Centre	Command classes, agents, approvals	9–12
Otic administration	Tenancy, metering and billing	4, 18
Legal assurance	Sources, conflict, provenance, ethical walls	10–15
Quality and release	UAT, resilience, sign-off	16–22

24. Open decisions and formal sign-off

This document is a technically actionable design proposal, not an assertion that implementation choices, service targets or legal-source rights have been approved. Final architecture approval requires product owner, lead engineer, security lead, legal-domain reviewer, operations lead and commercial owner to sign off relevant decisions.

Role	Decision scope	Status
Product owner	BRD/FSD scope alignment and release gates	Pending
Engineering lead	Architecture, stack, APIs, sizing and migration	Pending
Security / privacy lead	Threat model, isolation, retention and incident policy	Pending
Legal domain lead	Jurisdiction, source licensing, professional review	Pending
Commercial lead	Licensing, metering, cost and billing rules	Pending
QA / operations lead	Test evidence, SLOs, recovery and handover	Pending

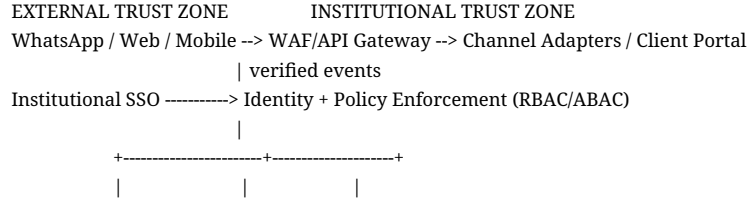
PART II | ENTERPRISE ENGINEERING IMPLEMENTATION BASELINE

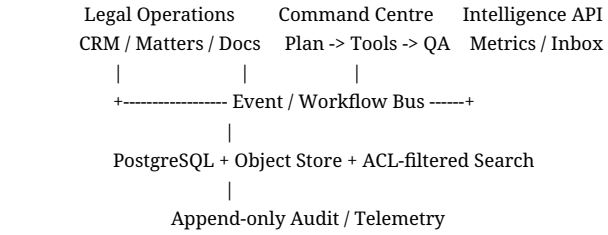
Version 1.1 enhancement. This implementation supplement is read with Sections 1–24 above. It elaborates agreed BRD v2.1 / FSD v1.1 functionality without introducing new product modules. All stack choices and numeric thresholds below are proposals, not institutional commitments.

25. Reference technology stack and selection criteria

Layer	Recommended candidate	Alternatives / decision criteria
Customer and institutional web	TypeScript, React / Next.js; accessible component system	Select for team expertise, accessibility, security and maintainability
Backend domain services	TypeScript/NestJS or Python/FastAPI; OpenAPI-first	Prefer one principal runtime for MVP; Python where AI tooling requires it
Workflow orchestration	Durable workflow engine / job queue with persisted state	Must support retries, timers, approvals and replay
Agent orchestration	Graph-based orchestration with explicit state transitions; e.g., LangGraph	Evaluate auditability, tool policies, model independence
Relational system of record	PostgreSQL with tenant-scoped policies and migrations	Dedicated databases for high-assurance deployments
Search / RAG	Permission-filtered full-text and vector search	Compare PostgreSQL extensions vs dedicated managed index
Files and evidence	Encrypted object storage with versioning, checksum and malware scanning	Residency, lifecycle and legal-hold support
Messaging	Official WhatsApp Business Platform through approved provider	Webhook authenticity, templates, costs and regional availability
Identity	OIDC provider supporting SSO, MFA, client verification	Enterprise federation, provisioning, audit and cost
Infrastructure	Containerised workloads, IaC, managed secrets and observability	Choose provider and region after residency/cost assessment

26. Detailed architecture and trust-boundary blueprint





Mandatory trust boundaries: public channel to ingestion gateway; authenticated user to domain APIs; domain service to AI tool runner; AI retrieval to tenant documents; tenant legal plane to Otic commercial plane. Every crossing requires identity, policy and structured logging.

27. Canonical sequence diagrams and failure branches

```
WhatsApp Provider -> Webhook Gateway: signed event(provider_event_id)
Gateway -> Dedup Store: atomic insert(provider, event_id)
  [duplicate] -> Gateway: return previously accepted outcome
Gateway -> Identity Service: verified client mapping / provisional profile
Gateway -> Intake Workflow: canonical enquiry + consent metadata
Workflow -> Event Bus: EnquiryReceived(enquiry_id, tenant_id, source)
Event Bus -> Unified Inbox: create or update projection
Event Bus -> Analytics: update first-touch cohort and processing watermark
Advocate -> Matter API: request conversion
Matter API -> Conflict Service: check and approval disposition
  [unresolved conflict] -> Advocate: blocked + escalation
  [cleared] -> Matter Store: create matter with enquiry lineage
Matter Store -> Audit: append clearance, actor, timestamp and event IDs
```

```
User -> Command API: question + explicit matter context
Command API -> Policy: actor, tenant, scope, allowed tools
  [deny] -> User: safe denial + audit
Planner -> Scoped Retrieval: legal sources and authorised records
Planner -> Specialist Agent: validated tool calls with budgets
Verifier -> User: cited answer / draft / action preview
  [action] -> Approval Service: payload hash, approver, expiry
  [approved] -> Domain API: idempotent commit with receipt
  [rejected/expired] -> User: no state change, reason recorded
Telemetry -> Metering: tenant usage units, cost and run correlation
```

28. Database schemas, constraints and indexes

Illustrative PostgreSQL DDL establishes minimum referential and idempotency patterns. Production schemas require detailed field review, retention policies, migrations and security testing.

```
CREATE TABLE tenant (id UUID PRIMARY KEY, mode TEXT NOT NULL, status TEXT NOT NULL);
CREATE TABLE enquiry (
 id UUID PRIMARY KEY, tenant_id UUID NOT NULL REFERENCES tenant(id),
 source_channel TEXT NOT NULL, client_id UUID, received_at TIMESTAMPTZ NOT NULL,
 stage TEXT NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE INDEX enquiry_tenant_channel_time ON enquiry(tenant_id, source_channel, received_at);
CREATE TABLE inbound_event (
 id UUID PRIMARY KEY, tenant_id UUID NOT NULL REFERENCES tenant(id),
```

```
 provider TEXT NOT NULL, provider_event_id TEXT NOT NULL,
 enquiry_id UUID REFERENCES enquiry(id), received_at TIMESTAMPTZ NOT NULL,
 UNIQUE(provider, provider_event_id));
CREATE TABLE matter (
 id UUID PRIMARY KEY, tenant_id UUID NOT NULL REFERENCES tenant(id),
 enquiry_id UUID REFERENCES enquiry(id), client_id UUID NOT NULL,
 status TEXT NOT NULL, clearance_state TEXT NOT NULL, version INT NOT NULL DEFAULT 1);
CREATE INDEX matter_tenant_status ON matter(tenant_id, status);
-- Enforce tenant scoping at application + database layers; composite FKs
-- and RLS policies must be verified in the final schema and test suite.
```

Further physical tables: client_identity, party_relation, conflict_check, conflict_decision, matter_member, ethical_wall, document, document_version, evidence_event, legal_source, legal_authority, command_run, tool_execution, approval, immutable_audit_event, usage_event, price_book and invoice_line. Use tenant-consistent composite keys or validated tenant checks on all cross-entity relationships.

29. API contract exemplars and error taxonomy

```
POST /v1/commands/runs
Authorization: Bearer <access_token>
Idempotency-Key: <unique-request-id>
{ "matter_id": "<uuid>", "command_type": "analysis",
  "query": "Show WhatsApp lead conversion this month",
  "requested_output": "chart_and_table" }

202 Accepted
{ "run_id": "<uuid>", "state": "queued", "correlation_id": "<uuid>" }

GET /v1/commands/runs/{run_id}
200 OK
{ "state": "completed", "result": {"metrics": [], "sources": []},
  "data_as_of": "<timestamp>", "warnings": [], "audit_ref": "<id>" }
```

Code	Meaning	Client action
AUTH_REQUIRED	No valid session or verification	Authenticate or verify
ACCESS_DENIED	Tenant, role, matter or ethical-wall restriction	Do not disclose resource existence
VALIDATION_FAILED	Schema or business-rule violation	Correct fields; preserve field errors
CONFLICT_PENDING	Matter clearance unresolved	Escalate to authorised reviewer
APPROVAL_REQUIRED	Consequential command not approved	Present preview to approver
SOURCE_UNVERIFIED	Citation source cannot be verified	Mark draft; require human review
DEPENDENCY_UNAVAILABLE	External provider or service failed	Retry only if safe; show status
RATE_LIMITED	Quota, abuse or budget threshold reached	Return retry policy or escalation

30. Agent state graph, model gateway and safety

State	Entry / exit contract	Failure transition

RECEIVED	Valid actor, tenant, query and correlation ID	REJECTED if invalid
SCOPED	Matter context and allowed resources resolved	DENIED on policy failure
PLANNED	Tool plan, model route, budget and stop conditions frozen	CLARIFY on ambiguity
RUNNING	Tools called with typed schemas, bounded retries and permissions	RETRYABLE_ERROR or FAILED
VERIFYING	Citations, calculations, jurisdiction, source dates and output schema checked	NEEDS_REVIEW if unsupported
AWAITING_APPROVAL	Action preview and immutable payload hash presented	EXPIRED or REJECTED
COMPLETED	Result/receipt persisted, usage metered and audit recorded	No silent partial success

Model gateway responsibilities: approved provider registry, per-tenant allowed models, input/output token caps, timeout policy, cost metering, PII minimisation, deterministic tool permission checks and controlled fallback. Do not rely on the model itself to enforce authorisation. AI provider training/data retention terms require legal and security approval.

31. Threat model and verification catalogue

Threat	Control	Required evidence
Cross-tenant data leakage	RLS + service-layer tenant binding + index ACL filters	Automated adversarial cross-tenant retrieval tests
Prompt injection in uploaded legal documents	Untrusted-source segmentation, tool allowlist, policy outside prompt	Injection corpus and prohibited-tool execution tests
Unauthorised matter search	Ethical-wall ABAC on query, retrieval and export	Denied search, RAG and aggregate drill-down tests
Webhook spoofing/replay	Signature, freshness, event deduplication	Forged, expired and repeated payload tests
Malicious upload	Scan/quarantine, MIME validation, object access policy	Infected file rejected before indexing
Citation fabrication or outdated law	Verified authority resolver and effective-date metadata	Benchmark of citation precision and freshness
Privilege escalation	MFA, scoped tokens, least privilege, dual control	Role escalation and approval-bypass tests
Audit manipulation	Append-only events, restricted write path, integrity checks	Audit integrity and clock correlation checks

32. Deployment modes and data portability

Mode	Deployment and isolation	Operational implication

Shared SaaS	Logically isolated tenants, shared managed services	Lowest marginal cost; strict tenancy controls
Dedicated tenant	Dedicated database/index/storage or full isolated stack	Higher assurance and higher unit cost
Institutional private cloud	Institution-controlled tenancy, networking and keys	Joint support and patching responsibilities required

All modes must use the same versioned product contracts and configuration engine. Institutional onboarding must support validated import templates, field mapping, deduplication, checksum reconciliation and migration reports. Offboarding must support authorised exports, key disposition, retention/hold exceptions and signed deletion attestations where legally permitted.

33. Capacity, service objectives and unit economics

The following sizing variables are mandatory inputs to a load and cost model, not fabricated forecasts: active tenants, users/tenant, daily conversations, peak messages/minute, documents/month, average pages/document, stored GB, vector-index size, daily AI runs, tokens/run, peak concurrency and recovery tier.

Cost driver	Metering unit	Margin / governance treatment
Model inference	Input/output tokens, model and tenant	Budget caps, routing, pass-through or bundled pricing
Document processing	Pages and processing minutes	Chargeback and anomalous-usage alerts
Messaging	Provider message/conversation charges	Channel-specific cost allocation
Storage / retrieval	GB-month, requests and index footprint	Retention-driven cost review
Compute / support	Allocated CPU/memory and operational labour	Tenant contribution margin and shared overhead

Gross contribution margin = (recognised tenant revenue − attributable variable AI, messaging, processing, storage, compute and support costs) ÷ recognised tenant revenue. Financial reporting must distinguish estimates from invoiced/reconciled charges.

34. Operational runbooks and release compatibility

Runbook	Minimum procedure	Owner to assign
Failed messaging webhook	Inspect signature, replay key, queue lag, dead letter and provider status	Integration operations
Agent failure	Inspect run graph, tool trace, model gateway, source validity and cost cap	AI platform lead
Tenant data incident	Contain tenant access, preserve evidence, assess exposure and	Security lead

	notify per policy	
Database recovery	Verify backup, restore in isolated environment, replay events, reconcile checksums	Platform operations
Bad release	Disable feature flag, rollback application, manage schema compatibility	Release engineer
Tenant offboarding	Validate export, revoke keys, respect holds and document deletion	Tenant operations

Compatibility policy: version APIs, tool schemas, prompts, agent graphs, workflows, tenant configuration and database migrations. Use expand/migrate/contract schema rollout. Record per-tenant enabled versions, rollout cohort, rollback limits and post-deployment smoke tests.

35. Engineering verification matrix

Gate	Evidence required	Release blocker
G1 Architecture	Approved ADRs, threat model, deployment diagram, hosting/data-region decision	Unresolved critical data residency or trust boundary
G2 Contracts	OpenAPI, event schemas, ERD, role matrix and policy tests	Undocumented write API or cross-tenant access
G3 AI assurance	Agent graphs, tool contracts, evaluation datasets, legal source licence and citation tests	Unreviewed legal outputs released externally
G4 Performance	Load profile, p95 latency, queue behaviour, RTO/RPO drills	Targets absent or critical tests fail
G5 Business UAT	BRD→FSD→TSD→test mapping, UAT-21–32 evidence, KPI reconciliation	Unmet critical acceptance case
G6 Operations	Monitoring, alert routing, runbooks, restore drill, cost reconciliation	No accountable owner or recovery plan

36. Architecture decisions requiring explicit approval

Decision	Options / proposed baseline	Decision owner
ADR-09 Runtime and framework	TypeScript service core; Python AI workers only as needed	Engineering lead
ADR-10 Durable workflow	Persistent workflow engine with approvals and retry semantics	Platform lead
ADR-11 Deployment modes	Shared SaaS baseline; dedicated/private-cloud configuration	Product + security

ADR-12 Data migration	Versioned import/export and reconciliation contracts	Data lead
ADR-13 Model routing	Pluggable gateway with tenant-level allowed provider list	AI + security
ADR-14 Observability and audit	Trace IDs, append-only security events, redacted operations logs	Security + operations
ADR-15 Release governance	API/tool/prompt/schema versioning and reversible rollout	Engineering + QA

37. Final sign-off statement

TSD v1.1 is the consolidated technical design proposal aligned to BRD v2.1 and FSD v1.1. It becomes the engineering implementation baseline only after architecture decisions, jurisdiction/source rights, non-functional targets, provider selections, security controls and release evidence are formally approved. No unapproved numerical SLO, pricing, hosting region or technology choice is represented as final.